

CURSO 4: TALLER PRÁCTICO & PROYECTO FINAL INTEGRADOR

CLASE 04 • NIVEL 4 - INTEGRADOR

Clase 04: Desarrollo del Frontend: Dashboards con Streamlit

«Streamlit como el Salón de Control Visual para tu Backend de Python»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 04** del **Curso 4: Taller Práctico & Proyecto Final Integrador**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.



Objetivos de la Sesión

Competencia Conceptual: Comprender el modelo de ejecución reactiva de Streamlit, gestión de estado con `st.session_state` y conexión a APIs.

Competencia Práctica: Construir una interfaz gráfica moderna con formularios, gráficos y visualización de datos en tiempo real.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 04: Desarrollo del Frontend: Dashboards con Streamlit

Streamlit permite transformar scripts de Python en aplicaciones web interactivas para ciencia de datos e Inteligencia Artificial.

☀ Metáfora Central: Streamlit como el Salón de Control Visual para tu Backend de Python

Es como un tablero de mandos de automóvil donde cada botón y pantalla se conecta directamente al motor de tu backend.

Principios Teóricos y Modelo Mental

Modelo Reactivo: Cada vez que el usuario interactúa con un control (botón, slider), Streamlit reejecuta el script de arriba a abajo.

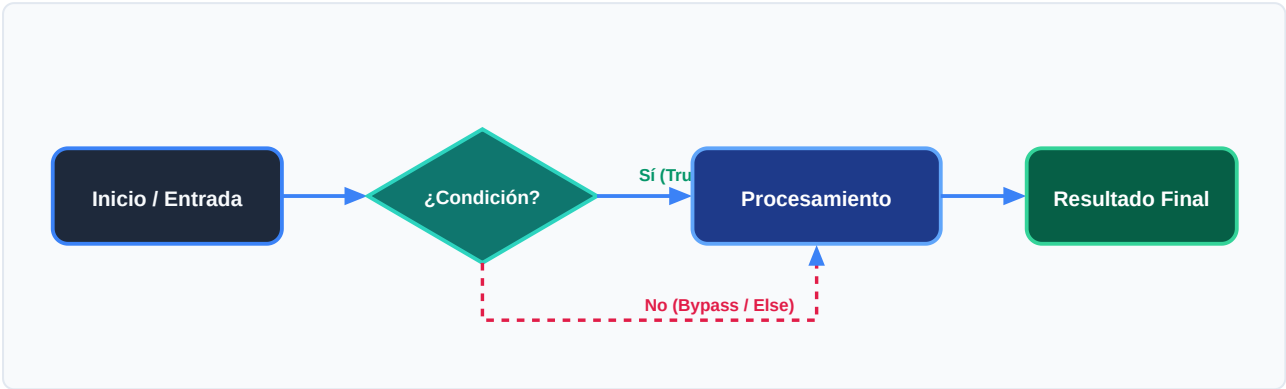
`st.session_state`: Permite preservar variables y estados entre reejecuciones del script.

⚡ Regla de Oro en Python

Usa `st.session_state` para almacenar sesiones de chat o datos de formularios sin perderlos al hacer clic.

Arquitectura y Movimiento de Datos en Memoria

Ciclo reactivo de eventos en Streamlit e invocación a la API.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Renderizado de componentes visuales (título, inputs).	DOM generado en el navegador.
2. Evaluación	Interacción del usuario (clic en botón).	Evento disparado hacia el servidor.
3. Transformación	Petición HTTP hacia el backend FastAPI (requests.post).	Respuesta JSON recibida.
4. Retorno / Salida	Actualización de st.session_state y mensaje de éxito.	Interfaz actualizada instantáneamente.

Visualización Mental

Todo lo que guardes en st.session_state sobrevive a la recarga de página.

Código de Demostración en Python 3.10+

Aplicación interactiva con historial en Streamlit:

main.py (Python 3.10+)

PEP 8 Compliant

```
import streamlit as st

st.set_page_config(page_title="Panel de Control", page_icon="🚀")
st.title("🚀 Panel de Gestión de Leads")

if "leads" not in st.session_state:
    st.session_state.leads = []

with st.form("form_lead"):
    nombre = st.text_input("Nombre completo")
    email = st.text_input("Correo electrónico")
    enviado = st.form_submit_button("Guardar Lead")

    if enviado and nombre:
        st.session_state.leads.append({"nombre": nombre, "email": email})
        st.success(f"Lead {nombre} registrado con éxito.")

st.write(f"Total registrados: {len(st.session_state.leads)}")
```

Análisis de Ingeniería del Código

Uso de `st.form` para agrupar inputs y `st.session_state` para persistencia de sesión.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Errores de rendimiento por recarga continua.

⚠ Gotcha Frecuente (Trampa de Principiante)

Cargar modelos pesados o archivos grandes en cada interacción ralentiza la aplicación.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
modelo = cargar_modelo_pesado_2gb() # ❌ Se  
recarga en cada clic
```

✅ Patrón Correcto

```
@st.cache_resource  
def get_model(): return cargar_modelo() # ✅ Se  
ejecuta una sola vez en caché
```

🛡 Consejo de Resiliencia en Producción

Usa `@st.cache_data` para llamadas a APIs y `@st.cache_resource` para objetos de conexión.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 04: Desarrollo del Frontend: Dashboards con Streamlit**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Streamlit como el Salón de Control Visual para tu Backend de Python
2. Regla de Oro	Usa <code>st.session_state</code> para almacenar sesiones de chat o datos de formularios sin perderlos al hacer clic.
3. Gotcha Crítico	Cargar modelos pesados o archivos grandes en cada interacción ralentiza la aplicación.
4. Buenas Prácticas	Usa <code>@st.cache_data</code> para llamadas a APIs y <code>@st.cache_resource</code> para objetos de conexión.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Crea una vista con `st.tabs` para alternar entre el formulario de registro y la tabla de datos.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.